AI Chat Messaging Application - POC Report

1. Objective

To develop a web-based AI Chat Messaging application where users can send messages via a frontend interface,
the backend processes the request using an AI API, and responses are displayed in real time.
Chat history is stored in MongoDB for persistence.

2. Technology Stack Used

Backend: Python (Flask)
Frontend: HTML + JavaScript
Database: MongoDB Atlas
AI API: OpenAI Chat Completion API
Environment: Python Virtual Environment (venv)

3. APIs Attempted

OpenAI Chat Completion API was integrated using the official Python SDK.
Model Attempted: gpt-4o-mini
Purpose: Generate conversational AI responses dynamically.

4. Steps Followed

• Installed Python 3.11 and created virtual environment.
• Installed required libraries: flask, openai, pymongo, python-dotenv.
• Created MongoDB Atlas cluster and configured database access.
• Developed Flask backend with /chat endpoint.
• Integrated OpenAI API for AI-generated responses.
• Implemented frontend chat interface using HTML and JavaScript.
• Stored chat history in MongoDB.

5. Issues Encountered

During API testing, the following error occurred:

Error Code: 429
Type: insufficient_quota

Cause: OpenAI API requires active billing and available quota for successful request processing.

6. Current POC Status

✔ Backend functioning successfully
✔ MongoDB integration completed
✔ Frontend chat interface operational
✔ OpenAI API integration implemented in code
■ Live AI responses pending billing activation

7. Next Steps

• Activate OpenAI billing (upon approval)
• Perform full end-to-end testing
• Improve frontend styling and UX
• Add enhanced error handling
• Prepare deployment-ready version